



A Survey on Task Offloading in Multi-access Edge Computing

Akhirul Islam ^{a,*}, Arindam Debnath ^a, Manojit Ghose ^a, Suchetana Chakraborty ^b

^a Department of Computer Science and Engineering, IIIT Guwahati, India

^b Department of Computer Science and Engineering, IIT Jodhpur, India

ARTICLE INFO

Keywords:

Multi-access edge computing
Task offloading
Mobile edge computing
Survey

ABSTRACT

With the advent of new technologies in both hardware and software, we are in the need of a new type of application that requires huge computation power and minimal delay. Applications such as face recognition, augmented reality, virtual reality, automated vehicles, industrial IoT, etc. belong to this category. Cloud computing technology is one of the candidates to satisfy the computation requirement of resource-intensive applications running in UEs (User Equipment) as it has ample computational capacity, but the latency requirement for these applications cannot be satisfied by the cloud due to the propagation delay between UEs and the cloud. To solve the latency issues for the delay-sensitive applications a new network paradigm has emerged recently known as Multi-Access Edge Computing (MEC) (also known as mobile edge computing) in which computation can be done at the network edge of UE devices. To execute the resource-intensive tasks of UEs in the MEC servers hosted in the network edge, a UE device has to offload some of the tasks to MEC servers. Few survey papers talk about task offloading in MEC, but most of them do not have in-depth analysis and classification exclusive to MEC task offloading. In this paper, we are providing a comprehensive survey on the task offloading scheme for MEC proposed by many researchers. We will also discuss issues, challenges, and future research direction in the area of task offloading to MEC servers.

1. Introduction

In recent years mobile devices (MDs) have become an indispensable part of human lives. Rapid growth in sensing and communication technologies has recently opened up the possibilities of numerous applications towards realizing the true digital age [1]. With the increasing trend of smartphone usage, the usability of mobile devices (MD) is not limited to serve over-the-wire connectivity anymore. Smart applications for gaming, social media interaction, business, infotainment, or even the basic utilities heavily use the in-built sensors including camera and microphone to offer customized services. On-device processing and storage of these huge amounts of multimodal sensory data are not feasible considering the limited resources of the mobile devices. Hence, resource-intensive mobile applications such as real-time online games, virtual reality (VR) and augmented reality, image processing, voice, gesture or face recognition, or other similar user-adaptive services pose critical design challenges in achieving the desired performance for resource-constrained MDs [2].

Until now Mobile Cloud Computing (MCC) was considered to be an efficient solution to this problem [3] as the cloud servers have huge computation powers and storage resources [4], so offloading computation-intensive tasks to cloud servers reduces the burden of

MDs and save critical energy. However, cloud servers introduce a good amount of delay and jitter due to long propagation delay between MDs and cloud servers because of their geographical location [5]. Task completion time can also be increased further if the MDs offload all their tasks along with the generated data to the cloud by congesting the backhaul network. Multi-Access Edge Computing (MEC) is a new computing paradigm that has recently been introduced in practice to address these problems [6–8]. The basic principle of MEC is to bring the computation power of MCC to the edge of a network close to the MDs [9]. The applications running on MDs can now offload the compute-intensive tasks to the nearby MEC servers at low overhead, which can significantly alleviate the problem of network congestion and improve the response time for the running applications [10–12].

MEC servers are deployed at the edge of the network in cellular base stations or small cells or both [13]. A wide range of mobile applications nowadays tends to avail the MEC services for providing a better quality of experience (QoE) to the users. Emerging applications like distributed content delivery and caching, web performance enhancements, context-awareness, content optimization, and various smart city services in the domains of healthcare, transportation, energy, manufacturing, and

* Corresponding author.

E-mail addresses: akhirul.islam@iiitg.ac.in (A. Islam), arindam.debnath@iiitg.ac.in (A. Debnath), manojit@iiitg.ac.in (M. Ghose), suchetana@iiitg.ac.in (S. Chakraborty).

<https://doi.org/10.1016/j.sysarc.2021.102225>

Received 17 April 2021; Received in revised form 18 June 2021; Accepted 20 June 2021

Available online 24 June 2021

1383-7621/© 2021 Elsevier B.V. All rights reserved.

industrial automation [1,14] heavily rely on MEC infrastructure for task offloading.

The main objective of task offloading is to minimize the task execution time of applications running at UEs and to save energy consumption of UEs. The problem of efficient task offloading is multi-fold as MEC servers are not as resourceful as the cloud servers, so task offloading from UEs to the MEC servers has to be done in such a way that the resources of the MEC servers are utilized judiciously. The MEC servers hosted on different BS may not be equipped with the same set of services, and the computation load in the MEC servers may also vary dynamically with time. Optimal resource allocation for offloaded tasks, fair and stable load distribution among the MEC servers, and seamless coordination between the cloud and edge are only one facet of the problem to utilize the MEC services effectively [15]. The other challenge remains in understanding the dimension of the designated tasks to offload. Depending on the required QoE parameters for the running applications, the tasks can also vary in terms of priorities, progress, computing overhead, and other associated dependencies [16]. The selection, scheduling, and placement of tasks consist of the other set of challenges associated with the problem of task offloading. Thus the problem of optimal task offloading boils down to selecting the right task to offload to the right MEC server at the right time to meet the optimal performance of a running application while utilizing the MEC resources efficiently. This is clearly a multi-objective optimization problem and known to be NP-hard [17,18].

Task offloading in the context of our discussion can be defined as uploading a complete module of an application that includes the computation, required data, and other dependent libraries to a remote server and obtaining the result of computation from the remote server. While discussing the task offloading, it is also important to define scheduling as it is an essential part of task offloading. Scheduling is a method that executes a set of tasks (or subtasks) on a set of computing resources to optimize a goal [19]. One of the primary objectives of scheduling is to maximize the utilization of resources while minimizing the total task execution time [20]. In our discussion, scheduling can happen at UEs, the MEC server, or on the cloud in the case of collaborative MEC system models. A UE can either schedule a task for local execution or offloads it to a MEC server. A MEC server schedules the task for execution locally or forwards the task to the cloud or other MEC server depending on different system models. A similar concept as task offloading is computation offloading which means sending only the compute-intensive part of a module to a remote server and receiving the result of computation from the remote server.

The state-of-the-art literature is quite rich in providing various solution strategies to address the stated problem of MEC task offloading [21–23]. In the absence of standardization, the existing solutions vary widely in the assumption of architecture and system model, design of task model, consideration of the objectives, and use of an offloading mechanism to name a few. Over the last couple of years MEC has been emerging as a key enabling technology towards the realization of IoT and 5G visions [24–26]. MEC research lies at the intersection of mobile computing and wireless communications, which naturally opened up the possibilities of many research opportunities. In recent years, researchers from both academia and industry have significantly contributed to addressing various challenges related to MEC, including system and network modeling, resource allocation, implementation, and standardization.

MEC network paradigm is still not a mature technology, and research in this direction is also in the early stage. There are few survey publications [1,16,27–31] available related to MEC which mainly focused on MEC taxonomy, future research directions and more specific MEC attributes like communication, computation offloading, resource allocation, security, virtualization, and applications. Though these articles talk about computation offloading, they described the task offloading very briefly. In the survey [32], the authors have focused mainly on decisions on computation offloading, allocation of computing resources

within the MEC, and mobility management. They have classified the task offloading mainly on two parameters i.e. execution delay and energy consumption. Authors in [33], mainly focused on task offloading based on the multi-task learning approach. Authors in [34] have tried to classify the existing work on task offloading based on 1. gaming and cooperation between edge and the cloud and 2. heuristic based computation offloading. The survey [35] presents an overview of the task offloading technology with the collaboration of edges and clouds. Authors in [36] highlighted the importance of offloading modeling and explore future research trends in this area. The survey [37] reviews task offloading methods for edge computing environments into five categories i.e. offloading destination, load balance of edge servers, device mobility, application partitioning, and partition granularity. The authors in [38] have reviewed the computation offloading approaches based on game-theoretic approach for edge computing. A summary of the above survey papers is presented in Table 2. Most of the surveys mentioned above mainly focused on the breadth of different topics on MEC, but these literature lacks in-depth analysis and classification of MEC task offloading. We are trying to make an in-depth classification of task offloading and the architectural model used for the task offloading which is different from the above surveys. We also discuss the associated design challenges, practical use cases, and potential research directions in this field.

The rest of the paper is organized as follows. Section 2 presents a formal definition of the task-offloading problem in MEC and evaluation metrics for the task offloading. In Section 3 we describe different system models under three broad categories and Section 4 is about different offloading strategies. Real-life use cases of MEC are briefed in Section 5, while technical challenges and future research direction are described in Section 6 and finally, Section 7 concludes the whole paper. We have summarized the definitions of the acronyms that will be frequently used in this paper in Table 1 for ease of reference.

2. Task offloading

The principal objective of MEC lies in supporting the resource constrained mobile devices with extended cloud services at the edge of the network. This is particularly helpful for emerging delay-sensitive and high compute-intensive mobile applications as they can save critical resources through offloading few tasks to the nearby MEC hosts. The key objectives of task offloading are to improve QoE by reducing computational delay, to extend the UE lifetime by reducing device energy consumption [11], and to maximize the revenue for the network operators and service providers.

2.1. Task: Definition and model

A task can be defined as a basic unit of work to be executed towards the accomplishment of an application service. When an application runs on mobile devices, it accomplishes a series of tasks, sometimes in a loop, to provide the desired service. From a computer programming perspective, a task is a logical unit of a program containing a bunch of instructions tightly or loosely coupled to implement specific functionality. Depending on how the application program is developed, a task may be an entire program or a set of one or more functions in a program. Since one user program may avail the services of other utility programs, a utility program may also be considered as a separate task (or sub-tasks). For example, an operation like matrix multiplication, derivation of a complex function, evaluating a complex mathematical expression, searching, sorting, or executing a specific algorithm can be treated as a task.

A task $\mathcal{T}$, can be formally defined as a vector (I, S, C) , where I is the set of instructions, S is the input size in terms of bits, and C denotes the set of CPU cycles needed by the task. Each C_i represents the CPU cycle requirement for the corresponding instruction I_i in a task [39]. The order of instructions to be executed to accomplish a task

Table 1
Summary of important acronyms.

Acronym	Definition	Acronym	Definition
MD	Mobile Device	MNO	Mobile Network Operator
UE	User Equipment	ACCO	Approximation Collaborative Computation Offloading
MEC	Multi-access Edge Computing	SDN	Software Defined Network
MCC	Mobile Cloud Computing	DRL	Deep Reinforcement Learning
QoE	Quality of Experience	MRL	Meta Reinforcement Learning
QoS	Quality of Service	MBS	Macro Base Station
DAG	Directed Acyclic Graph	eNB	Evolved Node B
BS	Base Station	HGOS	Heuristic Greedy Offloading Scheme
SC	Small Cell	MIP	Mixed Integer Program
KKT	Karush–Kuhn–Tucker	MINLP	Mixed Integer Nonlinear Programming
IOSDTO	Iterative Optimization of Service Deploying and Task Offloading	BBM	Branch and Bound Method
RNIS	Radio Network Information Service	DPOA	Dynamic Programming based Offloading Algorithm
DTOS	Dynamic Task Offloading and Scheduling	IPM	Interior Point Method
MRLCO	Meta Reinforce Learning Computation offloading	MDP	Markov Decision Processes
VPN	Virtual Private Network	BnB	Branch and Bound
CPE	Customer Premises Equipment	VM	Virtual Machine
V2I	Vehicle-to-infrastructure	RAN	Radio Access Network
MOSP	Modified Online Saddle Point	CCC	Centralized Cloud Computing

Table 2
Some references of existing surveys in task offloading.

Paper	Brief summary
[32]	The authors in this survey have focused mainly on decisions on computation offloading, allocation of computing resources within the MEC, and mobility management. They have classified the task offloading mainly on two parameters i.e. execution delay and energy consumption.
[33]	The authors in this survey mainly focused on task offloading based on the multi-task learning approach
[34]	Tried to classify the existing work on task offloading based on 1. gaming and cooperation between edge and the cloud and 2. heuristic
[36]	This survey highlights the importance of offloading modeling and explores future research trends in this area. They have classified exiting offloading schemes into four categories 1. offloading from ED to EC 2. offloading from EC to cloud computing 3. offloading from one EC server to another (others) 4. hierarchical offloading
[35]	This survey presents an overview of the task offloading technology with the collaboration of edges and clouds. The main parameters they have considered while classifying the task offloading are response time, device energy, service provider cost, load balance between edge and cloud, and device mobility
[37]	This survey reviews task offloading methods for edge computing environments into five categories 1.offloading destination 2. load balance of edge servers 3. device mobility 4. application partitioning and 5. partition granularity
[38]	The authors have reviewed the computation offloading approaches based on game-theoretic for edge computing

is dependent on the system architecture and it is out of the scope of this paper. Other attributes associated with a task might include the priority, deadline, the list of associated applications, and the level of dependency. Generally, each task is assigned a unique task ID so that every task can be uniquely identified within a UE and mapped with the corresponding application. The level of dependency for a task defines its offloading capacity and a less dependent task can be seamlessly offloaded to the MEC server. However, it is important to be careful while scheduling a task for offloading without violating any operational constraints for high dependency level tasks. The dependency can be either bound to the system architecture or the execution of other tasks and a task that is closely dependent on the platform should not be offloaded. If the dependency is in terms of the level of association with other tasks, then care must be taken in scheduling the task while offloading.

Task-related dependency is quite common as the functional progress of a task is often dependent on the completion of one or more tasks.

Moreover, a task can be partitioned into multiple sub-tasks based on the complexity of the corresponding application design and desired performance characteristics. These sub-tasks may either execute independently or depend on other tasks based on the functionality and logic of control flow for the running application program.

Those tasks which are well integrated, highly coupled, or relatively simple and cannot be partitioned into multiple dependent or independent tasks, that has to be executed as a whole either locally at the mobile devices or offloaded to MEC servers called binary offloading [30]. Such a task can be represented by a three-field notation $\mathcal{T}(L, D, X)$. This commonly used notation contains the information of the task input data size L (in bits), the task completion deadline D (in seconds), and the computation intensity X (in CPU cycles per bit).

Therefore, a computation task can either be executed locally at the UE or be offloaded to a nearby MEC host for processing. The offloading decision is made and the offloading process is triggered by the running application.

2.2. Evaluation metrics for local execution

Let us define c^L be the current computing capability of the mobile device in CPU cycles per unit time, w_i be the CPU cycles needed in total to complete the task $\mathcal{T}_i$. Thus total execution time of the task, denoted by δ_i , can be expressed as

$$\delta_i = \frac{w_i}{c^L} \quad (1)$$

Also, we can obtain the energy consumption for local task processing as [40]:

$$\psi_i = \alpha w_i \quad (2)$$

where α is the coefficient denoting the consumed energy per CPU cycle.

2.3. Evaluation metrics for MEC-server-side execution

For task offloading to the MEC server, the application QoE is characterized by the primary objective of reducing the average task duration time with the limited battery capacity. Let us denote the computing resource (CPU cycles per unit time) of the designated MEC servers to be c^S . The total task duration, denoted by Δ_i for any task $\mathcal{T}_i$ can be defined as the total response-time for the task starting from its submission time till its completion. Hence, Δ_i is composed of the task dispatch time (T_d), the propagation time for uploading the task to the MEC server (T_u), the task execution time (T_e) and the propagation time for downloading the

task output to the UE (T_r). We are assuming the value of T_d is a very small and constant for all tasks. T_r can also be ignored, considering the output data size for the task to be negligible compared to its input [40–42]. Therefore, the total task duration for task $\mathcal{T}_i$ can be obtained as:

$$\Delta_i = T_u + T_e = \frac{S_i}{B_{up}} + \frac{w_i}{cS} \quad (3)$$

where B_{up} is the up-link data rate and S_i is the size of input data to be uploaded as part of the task $\mathcal{T}_i$. We obtain the energy consumption at an UE with transmission power ρ for offloading the task $\mathcal{T}_i$ to edge server is as follows:

$$\Psi_i = \frac{\rho(S_i + w_i)}{B_{up}} \quad (4)$$

where w_i is the extra data (metadata in bits) needed to carry the task offloading data to a MEC server for the task $\mathcal{T}_i$. The size of the metadata depends on different protocols used to transmit the task offloading data to MEC server.

2.4. Task offloading problem formulation

Based on the evaluation metrics defined in Sections 2.2 and 2.3, we are going to formulate the task offloading problem in this section. Let us define the integer decision variable, $x_i \in \{0, 1\}$, which indicates task $\mathcal{T}_i$ is processed locally ($x_i = 1$) or in MEC server ($x_i = 0$). Now, the objective function is to minimize the total latency for executing n such tasks. Mathematically, the problem can be formulated as below (similar to [36]).

$$\text{minimize} \sum_{i=1}^n [x_i \delta_i + (1 - x_i) \Delta_i] \quad (5)$$

subject to following constraints:

$$\sum_{i=1}^n x_i \Psi_i + \sum_{i=1}^n [(1 - x_i) \Psi_i + \varepsilon] \leq \mathbb{E}_{max} \quad (6)$$

The constraint (6) indicates the total energy consumption of a UE should be less than or equal to its remaining battery capacity $\mathbb{E}_{max}$. Here ε signifies the minimum amount of energy required to receive the task output from the MEC server. The task duration minimization problem (5) can be proved to be NP-hard [18].

Task offloading problems can have many stakeholders based on the MEC architecture like MDs, MEC servers, cloud, etc. Task offloading is a challenging problem to solve for the following reason:

1. It is a multi-objective optimization problem and known to be NP-hard which cannot be solved using traditional deterministic algorithms in polynomial time.
2. To solve this problem, look-ahead knowledge on the amount of arriving tasks at the MDs is required.
3. While formulating the task offloading algorithm, we need to consider wireless channel conditions from the MDs to the MEC servers and computing resources across the entire network.
4. The task offloading problem itself includes many subproblems like task scheduling, resource allocation, task offloading decision making, and so on, which are closely interrelated.
5. MEC servers are not homogeneous. Along with varying load, the variety in hosted services across the MEC servers creates a problem in host selection. The service mapping is conventionally a dynamic effort and may suffer from instability resulting in a sub-optimal solution.

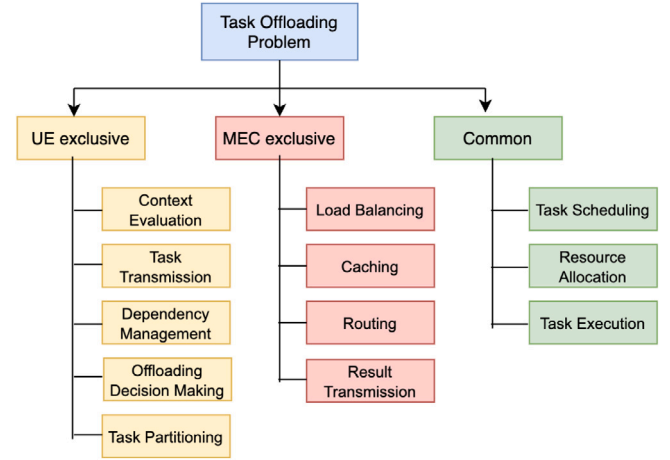


Fig. 1. Taxonomy of task offloading sub problems.

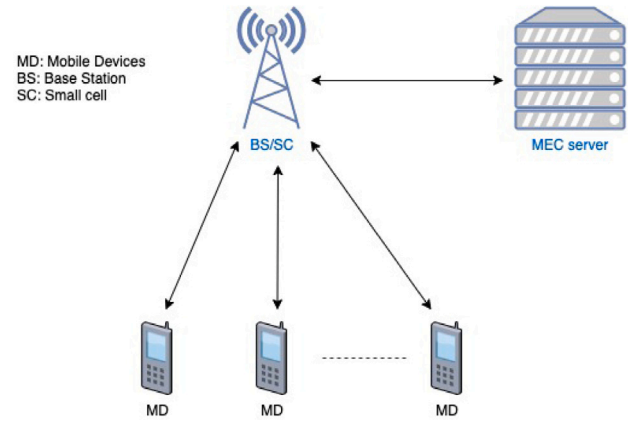


Fig. 2. Single BS with single MEC server.

2.5. Task offloading subproblems

Task offloading as a whole is a complex problem and it further can be divided into different subproblems. Some of the subproblems are exclusive to UE and MEC whereas some of them are common to both UE and MEC. Though discussing these subproblems in details are out of the scope of this literature, we have tried to categorize them as shown in Fig. 1. A state-of-art review of existing task offloading work, we have described in Section 4 shown in Fig. 5. In Section 4, different authors have tried to address one or more sub-problems provided in this section shown in Fig. 1.

3. MEC architecture classification

Although The European Telecommunication Standards Institute (ETSI) has proposed some generic reference architecture [43], there is no concrete standardized architectural framework for MEC till now to the best of our knowledge. In the absence of any concrete standardized architectural framework, diverse design practices can be observed across various MEC research communities in modeling the system architecture. The solution approaches proposed by different authors for MEC task offloading are mainly defined by the system model assumptions and reference architecture followed. After analyzing most of the recent research papers on MEC task offloading, we have categorized the MEC architecture into three broad classes as follows. We are assuming the MEC servers are hosted in the base stations only in the classification of MEC architecture.

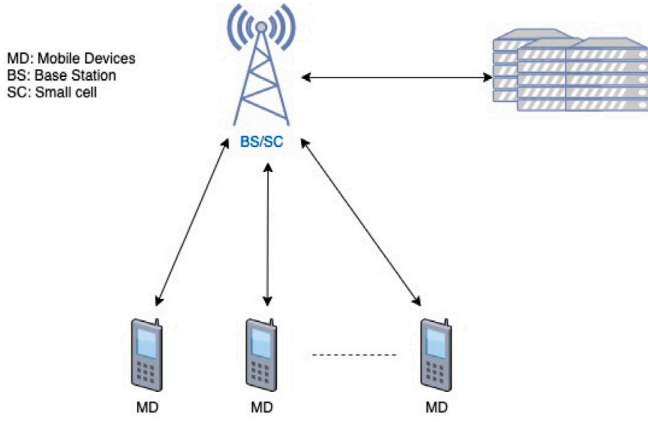


Fig. 3. Single BS with multiple MEC servers.

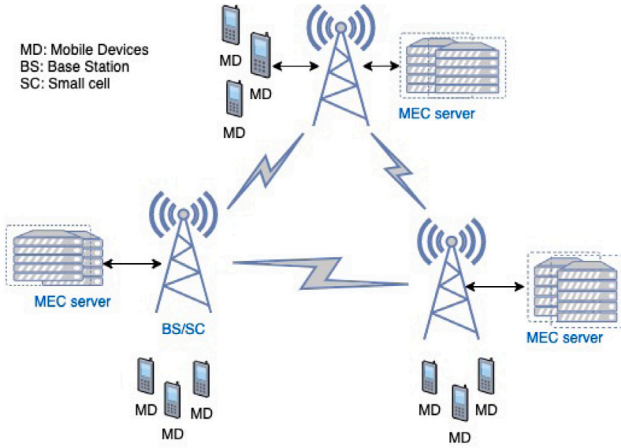


Fig. 4. Multiple BS with multiple MEC servers.

1. **Single MEC server in single BS/Small Cells:** As shown in Fig. 2, there exists one MEC server connected to a base station and MDs offload their tasks to the nearest BS based on the offloading decisions made by the UEs. In this architecture, only one MEC server serves the requests for all MDs connected to that BS and thus creating a high possibility of resource unavailability at the MEC server.
2. **Multiple MEC server with single BS/Small Cells:** As shown in Fig. 3, there exist multiple MEC servers connected to a single BS. MDs offload the tasks based on their task offloading policy to the nearest BS, and the tasks are distributed among all the MEC servers connected to that BS. There is less chance of server overloading as compared to the first model described above. In this architecture, no more offloading is done to other BS if the target BS is overloaded.
3. **Distributed MEC servers with multiple BS/Small Cells:** As shown in Fig. 4, there will be multiple MEC servers connected to a BS. MDs will offload the tasks based on their task offloading decisions to the nearest BS, and the tasks will be distributed among all the MEC servers connected to that BS or route the tasks to other MEC servers running on other BSs. In this architecture a task can be routed to other BSs depending on the following conditions (i) All the MEC servers in the target BS are overloaded (ii) The required application is not available in any MEC server

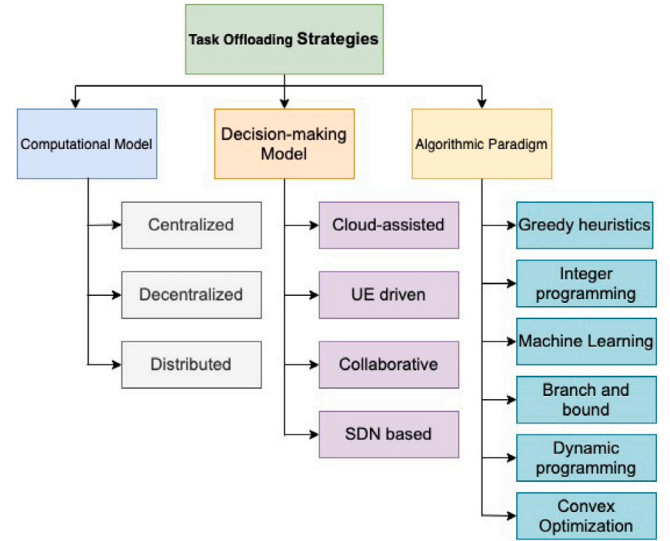


Fig. 5. Task Offloading Strategies (Section 4).

running on the target BS or (iii) If the task deadline cannot be met with the support from MEC servers in the target BS.

4. Offloading strategies

Task offloading is the key problem in realizing the MEC-enabled smart IoT applications. This is why it has got huge traction from both academia as well as the industry in terms of research contributions in the fields. The adopted architecture and system model widely vary across the MEC research communities coming up with diverse task offloading solutions. Hence, it is unfair to evaluate all these different solutions under one umbrella. As discussed in Section 2, the task offloading is an NP-hard problem, and we can only find an approximate solution to this problem. However, there will always be a trade-off between the performance gain achieved through offloading the task and offloading overhead incurred by the algorithm used.

The existing literature is widely diverse in terms of adopted architecture and system model assumptions, consideration of task model, use of algorithmic paradigm and computational strategies, and the task offloading problem formulation. We have broadly categorized different solution strategies into three different perspectives, which can further be sub-divided into various classes of problem-solving approaches taken as follows.

A detailed comparison of different works on MEC task offloading is depicted based on different categories and parameters in Table 3. We have tried to summarize the research works in the direction of MEC task offloading based on the different categories like system model, computation offloading model, evaluation methodology, the method used to solve the problem, etc. We have also tried to highlight some limitations of the different proposed solutions in different research works in Table 3. We have categorized only those works (tasks offloading on MEC) which are generic and not specific to a particular application like vehicular technology [62–73], UAV [74–76], Industry4.0 [77], or urban informatics [78] and others.

4.1. Computational perspective

A comparative analysis of different computational system models with respect to the performance benefits is shown in Table 4. In this section, we summarized different task offloading strategies with respect to different computational models. We have also presented this in Table 5 with a detailed insight.

Table 3
Task offloading detailed comparison.

Paper	Problems addressed	Method applied	Performance metrics	Evaluation methodology	Supported UEs	Computation offloading model	System model	Limitation
[44]	Task placement, Resource allocation	Nonlinear programming, SDTO	Task duration, Energy consumption	Simulation	Multi UEs	Mixed Offloading	Distributed	Ignores the delay and energy consumption of sending request from mobile device to the SD-UDN controller.
[45]	Task placement, Resource allocation	Greedy algorithm	Network delay, energy consumption	Numerical	Multi UEs	Full Offloading	Distributed	No consideration of the neighboring base stations which have MEC servers.
[46]	Task scheduling, Resource allocation	Mixed Integer Programming	Task admission and execution time	Numerical	Multi UEs	Full Offloading	Distributed	As the no. of UEs increases the admission rate decreases
[47]	Resource allocation	Greedy heuristic	Energy consumption	Numerical	Single UE	Partial Offloading	Centralized	Can only achieve a near-optimal solution.
[48]	Task offloading, Service deployment	BBM, IPM	Revenue of MNO	Simulation	Multi UEs	Full Offloading	Centralized	No fault tolerance and scalability
[49]	Joint task offloading, resource allocation	IPM, BBA	Energy consumption, Task processing delay	Numerical	Multi UEs	Mixed Offloading	Centralized	Not better than AOP in terms of Average task processing delay when task complexity is increased.
[50]	Task scheduling	Deep learning	Latency	Simulation	Single UE	Partial Offloading	Distributed	When operating at large-scale, some UE may drop out due to broken connection or insufficient power.
[15]	Collaborative computation offloading, task placement	Greedy, GT	Energy consumption	Simulation	Multi UEs	Full Offloading	Centralized	OECCO is unworkable for a large no. of computation tasks and has a very high computation complexity.
[41]	Task offloading and resource allocation	Greedy heuristic	Average time and energy consumption	Numerical	Multi UEs	Full Offloading	Distributed	Complexity of task offloading of the proposed scheme is high
[51]	Resource allocation, channel allocation, power control	Greedy heuristic	Energy consumption	Simulation	Multi UEs	Partial Offloading	Centralized	In case of splitting tasks, they have not mentioned how they are splitting the tasks.
[52]	Task partition, Task scheduling	Greedy Heuristic	Task execution latency	Numerical and Simulation	Multi UEs	Mixed Offloading	Distributed	For better performance high bandwidth and large numbers of MEC server needed
[53]	Task scheduling	Collaborative offloading	Latency	Simulation	Multi UEs	Full Offloading	Centralized	Average task failure increases as the task size increases for all the cases.

(continued on next page)

4.1.1. Centralized model

Centralized computing refers to a computing model involving a central server with high computing capability and sophisticated applications. The central server connects to client computers that have very low processing capability, and hence when a task needs to be executed, the clients simply send requests to the central server, which then performs most of the processing. The connection between the central server and clients can be either direct or over a network [83]. As

all requests are processed by a central server, the centralized computing model has lower efficiency and a higher risk of failure, compared to the decentralized or distributed computing model. However, the centralized model provides higher security and reliability with respect to user data, as all the data is stored in one place and tasks are performed on a single site reducing the chances of multi-party exposure [84,85]. Task offloading strategies proposed in [51] and [48] are based on a centralized computation model. In [51], a greedy sequential selection

Table 3 (continued).

Paper	Problems addressed	Method applied	Performance metrics	Evaluation methodology	Supported UEs	Computation offloading model	System model	Limitation
[54]	Dynamic task placement and Resource allocation	Distributed algorithm	Total energy-efficiency cost at UEs	Simulation	Multi UEs	Full Offloading	Centralized	Mao's Method [13] and Dinh's Method [13] are slightly better method when the number of tasks is small.
[55]	Task partition, placement, and Resource allocation	BnB, Iterative Heuristic	Latency	Simulation	Multi UEs	Partial Offloading	Distributed	Non-linear applications with complex internal dependencies are not considered.
[56]	Task allocation and Computational frequency scaling	Greedy algorithm	Total cost(energy consumption, execution delay)	Numerical	Single UE	Full Offloading	Distributed	Channel congestion may occur due to limited channel resources and higher the communication Cost.
[57]	Resource allocation	Sub-optimal resource allocation algorithm	Energy consumption	Simulation	Multi UEs	Full Offloading	Centralized	Not considered the effect of heterogeneous computation loads
[58]	Resource allocation	Sub-gradient algorithm	System delay	Numerical	Multi UEs	Partial Offloading	Centralized	Co-channel interference is not considered.
[59]	Task computation, Resource allocation	Heuristic algorithm	Total energy cost(UE, MEC system)	Simulation	Multi UEs	Full Offloading	Distributed	When the number of UEs grows the interference among UEs become severe which leads to a relatively high sum energy cost.
[60]	Task partition	Approximation	Energy consumption	Simulation	Single UE	Partial Offloading	Distributed	Codec energy model is not suitable for less complex local computations
[61]	Task offloading and resource allocation	Reinforcement learning	System cost(energy and delay)	Simulation	Multi UEs	Partial Offloading	Distributed	As resources are allocated dynamically based on the task arrival, it will be a challenging task when there is uncertainty in task arrival, and there is no central controller.

Table 4

Analysis Table for different computational model.

	Centralized	Decentralized	Distributed
Design complexity	Low	High	High
Scalability	Low	High	High
Horizontal scalability	No	Yes	Yes
Vertical scalability	Yes	Yes	Yes
Fault tolerance	Low	High	High
Single point of failure	Yes	Yes	Yes
Availability	Low	High	High
Design complexity	Low	High	High

algorithm **SMSEF** (Select Maximum Saved Energy First) for task offloading is proposed. The main objective of this work is to maximize the energy saved for mobile devices. In the proposed solution a task can be divided into subtasks and the divided subtasks can be computed locally or remotely. The methodology followed in splitting the tasks as well as the feasibility analysis for performing such a splitting operation is not discussed in the paper though. In [48], the authors have emphasized maximizing the total revenue from the perspective of

the mobile network operator (MNO), while meeting the requirements of resource-constrained mobile devices. According to their proposed model different services can run on the virtual machines of the MEC server. The main goal is to efficiently use MEC resources to maximize the revenue of the MNO while assigning the offloaded task to one of the services associated with a virtual machine.

4.1.2. Decentralized model

In decentralized systems, every server makes its own decisions. The final behavior of the system is defined by the aggregated decisions made through a synchronized cooperation among the participating servers [86,87]. This type of system is highly available, has high fault tolerance, and is not a single point of failure. Designing this kind of system is a big challenge as it involves associated challenges like replica management and consistency enforcement across all copies of data distributed over multiple servers. A decentralized dynamic computation offloading framework based on the deep deterministic policy gradient (DDPG) [88] algorithm has been built, which enables each user to independently learn efficient offloading policies from only local observations for dynamic power allocation in a continuous domain [82].

Table 5

Task offloading comparison of papers based on computational model.

Computational model	Design objective	Reference	Proposed solution
Centralized	Minimizing energy consumption for MDs	[51]	A greedy Select Maximum Saved Energy First (SMSEF) algorithm.
	Maximizing the total revenue	[48]	The iterative optimization of service deploying and task offloading (IOSDTO)
	Minimizing the total delay in finishing the tasks	[79]	Deep reinforcement learning-based joint task offloading and bandwidth allocation.
Distributed	Minimizing total energy consumption while satisfying the maximum execution latency constraints	[15]	Cloud-MEC Optimal Collaborative computation Offloading
	Minimize the network delay	[45]	Distributed bandit optimization (DBO) algorithm
	Minimal total latency	[50]	MRL(Meta Reinforce Learning) based method MRLCO (Meta Reinforce Learning Computation offloading)
	Minimizing latency	[80]	Deep reinforcement learning (DRL) based task offloading in MEC
	Minimize the energy consumption at the mobile devices	[49]	Improved branch and bound algorithm
	Reduce latency	[46]	Dynamic Task Offloading and Scheduling problem (DTOS) as a Mixed Integer Program (MIP) (DTOS-MIP)
	Minimize the average task duration with the limited battery capacity	[44]	Software defined Task offloading(SDTO)
	Minimizing energy consumption for MDs	[47]	Heuristic greedy offloading scheme
	Minimize the time consumption and reduce the number of task failure	[53]	Collaborative Offloading
	Minimize the processing and transmission delay and reduce the power consumption for these two operations	[61]	Reinforcement learning based state-action-reward-state-action (RL-SARSA) algorithm
Decentralized	Maximization of offloading efficiency subject to task deadline constraints	[81]	The optimal mobile edge server selection strategy. algorithm.
	Minimize long-term average computation cost of MDs	[82]	Decentralized dynamic computation offloading.

While proposing the solution [82], authors have considered multi-input multi-output (MIMO) enabled multi-user MEC system with stochastic wireless channels. A peer-to-peer enabled decentralized edge server management scheme for edge computing environment is proposed in [81] by the authors. The task offloading scheme based on this model involves maximizing the offloading efficiency by selecting the optimal MEC server, subject to meeting the task deadline constraint is proposed by the authors.

4.1.3. Distributed model

Following the distributed computing paradigm, multiple MEC server entities communicate and coordinate with each other to appear as a single coherent system to the end-users. There are two general ways in which the distributed systems function: (1) each node works toward a common goal and the end-user views the results as one cohesive unit. (2) each node has its end-user and the distributed system facilitates sharing resources or communication services. Typically in this paradigm, a controller distributes the tasks to different servers based on various criteria like application type, latency deadline, types of resource requirement, and other parameters [89,90]. The distributed cluster can reside in a single BS or multiple BSs based on the system model. A distributed system is highly available and can scale both horizontally and vertically. Although there are many advantages of a distributed system, on the other hand, it has some design challenges such as heterogeneity, transparency, fault tolerance and failure management, scalability, concurrency, openness and extensibility, migration and load balancing, and security [91].

Task offloading schemes proposed in [15,44–47,49,50,53,80] are based on the distributed computing paradigm. The system model used in [15,45,49,50,80] consists of multiple computing servers in a single base station. A task offloaded to a base station must be computed on the servers belonging to that BS. But the system model used in [44,46,47] has the provision of redirecting the tasks to the servers of another base station if the task requirements cannot be satisfied by the MEC servers residing in the current base station. Most of these works have

not addressed the problem of task distribution across the servers. One of the solutions performing approximation collaborative computation offloading (ACCO) [15] proposed task distribution based on a centralized controller. However, the work does not discuss how the central controller distributes the tasks among the servers. In [44], the authors have discussed the problem of task offloading for software-defined ultra-dense networks, which is based on software-defined network (SDN), and the SDN controller is the brain of the system and responsible for task distribution among the servers. Though the SDN controller has the overall view of the network which takes the task distribution decisions, the authors have not mentioned what are the statistics it collects and from which all entities for formulating the task distribution decision model.

4.2. Decision-making perspective

In this section, we have provided a detailed classification of existing task offloading works based on decision-making entities like UEs, cloud, SDN controller, etc. and we have also presented the same in Table 6.

4.2.1. Cloud-assisted

In this approach, all the task offloading decisions are made at the cloud servers. The MEC servers send information about computation resources, network topology, different application types, and their corresponding QoS requirements to a centralized cloud server. Similarly, all the UE devices send various task attributes to the cloud. On the other hand, the cloud server processes the information received from MEC servers and UE devices, formulates the task offloading decision policy, and sends them to UEs. In [80], a deep reinforcement learning (DRL) based offloading framework is integrated into the MEC system where UEs or mobile devices send the application DAGs (Direct Acyclic Graph) to a centralized cloud. A deep reinforcement learning (DRL) based task offloading decision model is formulated in the centralized cloud by training the collected data that includes application DAGs and the current environment of the MEC system.

Table 6

Task offloading comparison of papers based on decision model.

Decision model	Design objective	Reference	Proposed solution
Cloud-assisted	Minimizing latency	[80]	Deep reinforcement learning (DRL) based task offloading in MEC
UE driven	Minimize the network delay	[45]	Distributed bandit optimization (DBO) algorithm
	Minimizing energy consumption for MDs	[47]	Heuristic greedy offloading scheme
	Minimizing total energy consumption while satisfying the maximum execution latency constraints	[15]	Cloud-MEC Optimal Collaborative computation Offloading
	Minimizing energy consumption for MDs	[51]	A greedy Select Maximum Saved Energy First (SMSEF) algorithm.
	Minimize the energy consumption at the mobile devices	[49]	Improved branch and bound algorithm
SDN based	Reduce latency	[46]	Dynamic Task Offloading and Scheduling problem (DTOS) as a Mixed Integer Program (MIP) (DTOS-MIP)
	Minimize the average task duration with the limited battery capacity	[44]	Software defined Task offloading(SDTO)
Collaborative	Minimal total latency	[50]	MRL(Meta Reinforce Learning) based method MRLCO (Meta Reinforce Learning Computation offloading)
	Minimizing the latency	[92]	A joint iterative algorithm based on the Lagrangian dual decomposition
	Maximizing the total revenue	[48]	The iterative optimization of service deploying and task offloading (IOSDTO)

4.2.2. UE driven

In this case, UE itself formulates the task offloading decision policy and offloads the task accordingly. As per this strategy, UE takes responsibility for making the decision of offloading the tasks. It may use different algorithm design paradigms like greedy technique, dynamic programming, integer programming, and so on to formulate the decision model. Since UE is taking the offloading decision, it has to expend some of its computational resources for this purpose which may result in added energy consumption. Task offloading strategy in [15,45,47,49,51] are UE driven where each UE decides whether to offload the tasks to MEC server or cloud or compute locally. The task offloading decision in [45,47,49,51] are based on maximizing the energy savings at UEs or minimizing the energy consumption at the mobile devices. The limitations of these works come with not considering the CPU cycles needed for computing the energy requirements for the tasks under consideration. The energy-efficient task offloading algorithms in [47,49] also considers the delay constraint of a task while formulating the offloading decisions. One of the main objectives of the task offloading algorithm [45] is to minimize the network delay along with maximizing the energy savings at the UE side. An ultra-dense network is considered in [47] where MEC servers are attached to a macro base station (MBS) and small cell (SC) BS. In this architecture [47], the MEC servers are attached to both MBS and SCs which adds more complexity in selecting the MEC servers in the task offloading decisions.

4.2.3. SDN based

Software-defined networking (SDN) technology is an approach to network management that enables dynamic, programmatically efficient network configuration to improve network performance and monitoring, making it more like cloud computing than traditional network management [96]. SDN attempts to centralize the network intelligence in one network component by disassociating the forwarding process of network packets (data plane) from the routing process (control plane). The control plane consists of one or more controllers, which are considered to be the brain of the SDN network where the whole intelligence is incorporated [97].

In this approach, the task offloading decision is modeled at the SDN controller based on different information collected from UEs and MEC servers from different BS. The decision model is then broadcast to UEs periodically by the SDN controllers. The task offloading mechanism for MEC in [44,46] is based on SDN architecture. The task offloading model in [44] is based on an ultra-dense network on top of SDN architecture. A task can be forwarded to a MEC server on another base station

(eNB) if the current one is unable to satisfy the task requirement [46]. However, the main challenge lies in selecting a suitable MEC server without knowing if the current server can satisfy the request or a neighbor eNB has a server that can serve the request. The server load varies dynamically, and hence the server selection for task offloading is a time-critical decision. The current literature has not addressed that problem yet. The arbitrary selection of the MEC server running on another BS to redirect the offloaded task might often lead to the ping pong effect. In the work described in [44], the decision of forwarding a task to another base station is taken by the SDN controller. Though the SDN framework has many advantages like network flexibility, easy troubleshooting, it has its limitations when it comes to security, scalability, high availability, reliability, and elasticity issues [96]. The current literature is still in its nascent stage in addressing all these issues towards solving the MEC offloading problem through SDN-based solutions.

4.2.4. Collaborative

A collaboration among UEs, MEC servers, and the mobile cloud can be classified into three categories based on the participation in the task offloading decision-making processes like 1. combination of UEs, MEC servers, and the mobile cloud 2. MEC servers of different service providers. 3. MEC servers of different networks with the same service provider. UEs decide which base station to offload the task or to compute the task locally. A controller in the BS will decide which server to redirect the request based on the requirement of a task. It may also be possible that the controller can select a server in the neighbors' BS depending on the MEC system model used. The system model adopted in [48,50,53] is collaborative involving both UEs as well as MEC servers. In [53], authors have proposed two types of collaboration: one is the collaboration between mobile users and MEC server for reducing the number of task failures and another one is the collaboration between MEC server with cloud for reducing latency as well as executing a large number of computation tasks. The MRL(Meta Reinforce Learning) based method MRLCO (Meta Reinforce Learning Computation offloading) algorithm has two parts of which one is executed in UEs and the other in MEC servers. In this proposed method, the UEs collaborate with MEC and cloud for decision-making on task offloading. A joint problem of a cooperative computation task offloading scheme proposed in [92] where the authors have considered vertical cooperation among UE devices, MEC server nodes, and cloud server nodes, and the horizontal computation cooperation between edge nodes.

Table 7

Task offloading comparison of papers based on algorithmic paradigm.

Algorithmic paradigm	Design objective	Reference	Proposed approach
Greedy heuristics	Minimizing energy consumption for MDs	[51]	A greedy Select Maximum Saved Energy First (SMSEF) algorithm
	Minimizing energy consumption for MDs	[47]	Heuristic greedy Offloading scheme
	Minimizing total energy consumption while satisfying the maximum execution latency constraints	[15]	The Algorithm approximation collaborative computation offloading solution (ACCO)
	Minimizing total energy consumption while satisfying the maximum execution latency constraints	[93]	A heuristic swap matching based algorithm
	Minimize the task completion time	[94]	A heuristic algorithm
Integer programming	Maximizing the total revenue	[48]	Convex optimization and branch and bound are used to solve the MIP problem
	Reduce latency	[46]	Dynamic Task Offloading and Scheduling (DTOS)
	Minimize the energy consumption at the mobile devices	[49]	Improved branch and bound algorithm
	Minimize the average task duration with the limited battery capacity	[44]	MINLP problem converted to a convex problem and used the Karush–Kuhn–Tucker (KKT) condition to obtain an optimal solution
Machine learning	Minimal total latency	[50]	Dynamic Task Offloading and Scheduling problem (DTOS) as a Mixed Integer Program (MIP) (DTOS-MIP)
	Minimizing latency	[80]	Deep reinforcement learning (DRL) based task offloading in MEC
	Minimizing the total delay in finishing the tasks	[79]	Deep reinforcement learning-based joint task offloading and bandwidth allocation.
	Minimize the processing and transmission delay and reduce the power consumption for these two operations	[61]	Reinforcement learning based state–action–reward–state–action (RL-SARSA) algorithm.
Branch and bound	Maximizing the total revenue	[48]	Iterative optimization of service deploying and task offloading (IOSDTO)
	Minimize the energy consumption at the mobile devices	[49]	Improved branch and bound algorithm
	Minimizing latency and device energy consumption	[95]	Reformulation-Linearization Technique (RLT) based solution
Convex Optimization	Minimize the average task duration with the limited Battery capacity	[44]	MINLP problem converted to a convex problem and used the Karush–Kuhn–Tucker (KKT) condition to obtain an optimal solution

4.3. Algorithmic paradigm

In this section, we have classified existing works on task offloading under the heading algorithmic paradigm. A summary of this section is illustrated in Table 7.

4.3.1. Greedy heuristics

As already discussed in Section 2, task offloading is an optimization problem and it is NP-hard, we can use a greedy heuristic to solve it. A greedy heuristic algorithm generally tends to find a near-optimal solution without costing too much computation time or memory space. This algorithm does not need any complex derivatives or careful choice of initial values for building a solution space. [98]. The task-offloading algorithms proposed in [15,47,51] are based on greedy approaches. SMSEF (Select Maximum Saved Energy First) is a greedy sequential selection algorithm [51] that chooses the most energy-saving task at each iteration. The heuristic greedy offloading scheme (HGOS) [47] first calculates the task processing time based on local computation, computation at the MEC server connected to the MBS, and computation at the MEC server connected to the SC. In the second phase, it is decided where to offload the tasks based on the processing time. The main challenge in HGOS is to calculate the value of the processing time for a task concerning local, MEC, or cloud platform. The Algorithm approximation collaborative computation offloading solution (ACCO) [15] also follows a greedy approach with the lowest energy consumption by the MDs as the evaluation parameter. The computational complexity of ACCO is $O(n^2)$, where n is the number of computation tasks. For greedy algorithms to apply, we need prior knowledge about the optimization parameters and for the task offloading scenario these parameters are

(i) total CPU cycles needed for a task (ii) network delay from UEs to MEC server and (iii) time per CPU cycle for both UEs and MEC servers. To get all those information and process them, the UEs have to use the computation power which in turn consumes energy. It is also important for UEs to be agnostic about the MEC server information. Authors in [93], formulated the task offloading problem as an optimization problem and proposed a heuristic swap matching-based algorithm to solve this problem. Authors in [94] proposed a heuristic algorithm for task offloading to MEC server that aims to minimize the task completion time. This algorithm assigns priority to each UE device according to their local computation frequency and input data size. The device with higher priority will have precedence to occupy channels and offload the task to MEC server attached to a BS.

4.3.2. Integer programming

The objective of integer programming algorithms is to minimize or maximize a function subject to equality, inequality, and integer constraints [99]. Integer constraints restrict some or all of the variables in the optimization problem to take on only integer values. When there are integer constraints on only some of the variables, the problem is called a mixed-integer program (MIP). An integer linear program in the canonical form is expressed as below [100].

$$\begin{aligned} &\text{maximize } c^T x \\ &\text{subject to } Ax \leq b, x \geq 0, x \in \mathbb{Z}^n \end{aligned} \quad (7)$$

where c , b are vectors and A is a matrix with all entries to be integers.

The task offloading optimization problem can be formulated into a mixed-integer program with constraints like remaining energy of the UE devices, computation abilities of UEs and MEC servers, other

resources needed for computation of a task etc. Task offloading problem in [46,48] is formulated as Mixed Integer Program (MIP) while in [44,49] the problem is formulated as Mixed-Integer Nonlinear Programming (MINLP). A non convex mixed-integer optimization problem is formulated to solve task offloading in [101]. An improvement of a branch and bound algorithm is proposed to address the MINLP in [49] instead of transforming the MINLP to a relaxed optimization problem as a convex one, which can be solved by the interior point method (IPM). Although the IPM can find the optimal solution for a relaxed problem, it may not be practical from the implementation point of view in the MEC network. The MINLP problem in [44] is divided into two sub-problems as (1) computing resource allocation problem and (2) task placement problem. For resource allocation, the problem is proved to be a convex problem and authors have used the Karush–Kuhn–Tucker (KKT) condition to obtain an optimal solution. Based on the solution of the above sub-problem, they adopted the task placement algorithm to obtain the solution for this sub-problem. The MIP problem in [48] is solved by decomposing the problem into two dependent sub-problems as (1) task offloading problem and (2) service deployment problem. For task offloading it becomes a convex optimization problem when the decision of service deployment is given. On the other hand, when the offloading strategy is fixed, the problem turns into a 0–1 programming problem. The authors have solved the mentioned sub-problem using the branch and bound method (BBM) and the interior point method (IPM) respectively and named it iterative optimization of service deploying and task offloading (IOSDITO). Dynamic Task Offloading and Scheduling (DTOS) [46] problem is solved by decomposing it into a master problem (MP) which solves the task assignment and the application resource allocation problems, and multiple subproblems (SP) to resolve the task scheduling problem.

Perhaps the simplex method is effective for solving linear programs, but there is no single technique for solving integer programs. Instead, several procedures have been developed, and the performance of any particular technique appears to be highly problem-dependent. Methods to date can be classified broadly into three approaches as follows: (i) enumeration techniques, including the branch-and-bound procedure; (ii) cutting-plane techniques; and (iii) group-theoretic techniques [102]. Sometimes we may need to use the composite methods (more than one method combined) to solve the integer problem. Though we can solve the task offloading problems using integer programs, it is difficult to model and can be hard to solve.

4.3.3. Machine learning

Machine learning (ML) is a sub-field of artificial intelligence (AI), which contains all the approaches that allow machines to learn from data without being explicitly programmed [103]. Machine learning is closely related to computational statistics, which focuses on making predictions by using computational resources [104]. By leveraging complex mathematical and statistical tools, ML renders machines capable of performing independently intellectual tasks that have been traditionally solved by human beings. A model for task offloading decisions can be formulated by training the different data set from UEs and MEC servers. Since the formulation of a decision model using machine learning is computation-intensive and needs huge resources, it is better to run machine learning algorithms in the far cloud which does not have resource constraints. The UEs periodically send the information of its computation power, task-related information like CPU cycle requirements, latency deadline, etc. Similarly MEC servers also periodically send the information of their resource capacity, type of applications supported to the cloud. The machine-learning algorithm running in the cloud uses this information from UEs and MEC servers and formulates the task offloading decisions. The decision model formulated by the deep learning algorithm a subfield of ML in the cloud is sent back to UEs, which in turn takes the offloading decision based on that model. The good thing about machine learning is, it learns from

past experiences and keeps refining the model and which helps UEs to offload tasks more intelligently and efficiently.

A machine learning-based task offloading scheme is proposed in [50, 80]. Deep reinforcement learning (DRL) [61,79,80] and MRL (Meta Reinforce Learning) [50] based approaches for learning the best task offloading decisions also exists in the literature. In [50], the authors have proposed an idea to model the dynamic computation offloading process as multiple MDP (Markov Decision Processes)s, where the learning of offloading policies is decomposed into two parts: effectively learning a meta policy among different MDPs, and fast learning a specific policy for each MDP based on the meta policy. They have converted the offloading decision process as a sequence prediction process and designed a custom seq2seq neural network to represent the offloading policy. The MRLCO (Meta Reinforce Learning Computation offloading) aims to leverage the computation resources from both the UE and the MEC host for the training process. The proposed model in [80] is divided into three layers: application, MEC, and cloud. The cloud layer gathers application DAGs from mobile devices and conducts DRL training based on both the gathered DAGs and the environment. The offloading problem is then formulated as a MDP, and a new S2S (Sequence-to-Sequence) neural network is proposed to model the offloading policy. The S2S neural network is trained via the Proximal Policy Optimization algorithm [105], which has high training performance and stability.

Though there are many advantages, deep neural networks are often much harder to train than shallow neural networks. To use machine learning in task offloading, it is important to design a system properly keeping the following points in mind (i) whether the machine learning model to formulate in the central cloud or the MEC server. (ii) how the machine learning model gathers the relevant information (iii) how the formulated task offloading model is pushed to UEs.

4.3.4. Branch and bound

Branch and bound (BnB) is a general programming paradigm used to solve hard optimization problems. The complexity of these problems is generally in exponential order. Branching is the process of creating sub-problems, and bounding is to ignore partial solutions that are not better than the current best solution. BnB methodology can be a good candidate to solve many optimization problems easily for which direct methods of a solution either do not exist or insufficient [106]. Task offloading is an optimization problem with many constraints and it can be formulated into an integer or mixed-integer program. As discussed in Section 2, it is an NP-hard problem, we can relax some variables to solve this problem using the BnB algorithm. The task offloading problem in [48] is formulated as MIP while in [49] as MINLP. The IOSDITO algorithm for task offloading sub-problem discussed in 4.3.2 is solved using the BnB methodology. An improvement of a BnB algorithm is also proposed to address the MINLP problem in [48]. In [49], the authors proposed the task offloading problem as a MINLP optimization problem, which is NP-hard. They have introduced a relaxing solution that converts binary decision variables to real values and proposed a BnB-based solution. The authors in [95] have proposed a QoE-driven computation offloading algorithm based on the BnB technique, where they considered latency time and energy consumption of UEs as the main attributes to measure QoE. The main goal of this paper [95] is to enhance the quality of service experienced by users by minimizing the weighted sum of the total latency delay and device energy consumption.

4.3.5. Dynamic programming

Dynamic programming is an algorithmic design strategy that transforms a complex problem into a sequence of simpler problems to solve an optimization problem. Its efficiency has been proved in many areas of wireless communication for solving optimization problems within an acceptable bound on computation time, such as optimal path searching in data packets routing [107], resource scheduling and

allocation [108], power management [109,110], and so on. Dynamic Programming based Offloading Algorithm (DPOA) [107] is an adaptive offloading algorithm that can determine the offloading policy dynamically according to a changing wireless environment. By applying a dynamic programming technique to an existing task offloading algorithm (if suitable), we may reduce the time complexity of that algorithm. By reducing the time complexity of the task offloading algorithm, we can save the energy of mobile devices and reduce the task completion time. It can also be used to cope up with dynamic environments like varying channel bandwidth between mobile devices and base stations or small cell stations. However, dynamic programming-based solutions suffer from a few drawbacks (1) It needs a significant amount of memory to store the calculated result of every sub-problem without ensuring if the stored value will be utilized or not. As a result, it uses more energy and resources of mobile devices. (2) There is no general method to solve a DP (dynamic programming) problem and the method to solve a DP problem varies from problem to problem.

4.3.6. Convex optimization

A convex optimization problem is a problem consisting of minimizing a convex function over a convex set. Mathematically a convex problem can be represented as below

$$\min f(x)$$

such that $x \in \mathbb{S}$

where $\mathbb{S}$ is a convex set and $f(x)$ is a convex function over $\mathbb{S}$. A set $\mathbb{S}$ is convex if for all members $x, y \in \mathbb{S}$ and all $\theta \in [0, 1]$, we have that $\theta x + (1 - \theta)y \in \mathbb{S}$. Convex optimization is a subfield of optimization problem and a convex optimization can be solved in polynomial-time algorithms [111], though the optimization problem is in general a NP-hard [112,113]. The computing resource allocation problem [44] is proved as a convex problem and they used the Karush–Kuhn–Tucker (KKT) condition to obtain an optimal solution.

5. Use cases

As MEC server hosts at the edge of the network, which becomes suitable for numerous modern applications. We will discuss some typical use cases of the MEC network paradigm as below.

1. IoT Applications: IoT devices have less computation power and need to prolong battery lives, so it is important to offload the computation-intensive tasks for remote processing and retrieve the results once the processing completes [114]. It is also difficult for some IoT devices to act without the aid of an external entity, so these IoT applications have to obtain distributed information for computation [16]. Although all these requirements are supported by a distant centralized cloud that has high-performance computation capabilities and can collect distributed information. One of the main challenges of centralized cloud computing is to fulfill the latency deadline for a section of IoT applications which can be solved by offloading computation intensive tasks to a MEC server [115]. A MEC server can also act as an IoT gateway to aggregate the messages from IoT devices nearby. In addition, the MEC server can pre-process messages and send only meaningful messages to the cloud. It will help in reducing the congestion in the backhaul network and significantly reduces the communication and processing overhead [27,116].
2. Video Stream Analysis: Different applications come under video stream analysis such as vehicular license plate recognition, face recognition, and home security surveillance, etc. Generally, video analysis algorithms have high computation complexity, so it is preferable to offload the analysis tasks away from video-capturing devices [16]. If these processing tasks are to be handled in the central cloud, the video stream should be

routed to the core network [117], which will consume a great amount of network bandwidth due to the nature of the video stream. If we can perform these analysis tasks close to the source of the video stream, it will help in getting rid of network congestion caused by the video stream uploading and the latency of task transmission between the source device and distant cloud [118]. A MEC-based video analysis system is a suitable choice for this kind of application. A real-life example could be a factory surveillance system. In this system, a large number of cameras can stream the surveillance videos to the edge server directly. The edge servers pre-process and analyze the streaming video and alert the administrator for any emergency in real-time and send the filtered video frames to the cloud servers. Cloud servers process and analyze the filtered video frames from the edge servers. The edge servers in this case help in alerting the administrator in real-time and reduces the traffic in the backhaul network [119].

3. Distributed Content Delivery and Caching [14]: One of the most popular services of mobile networks is content, and especially video delivery. Thousands of video content items are uploaded daily to content provider's networks and stored in large volumes in the content provider's centralized databases. These contents are then transcoded from source format to final delivery format before distributed to multiple streaming servers. Despite best distribution efforts from content providers, many users may experience service interruptions because of buffering problems and jitter due to lack of content in the proximity of end-users. A smart distributed video caching at the edge using the MEC framework [120] by extending Content Delivery Network (CDN) services can enhance the quality of experience (QoE) of the users. This solution can also help in reducing backhaul and core network usage and congestion. The streaming service providers like Netflix,¹ Amazon Prime,² etc. can cache the popular content of a particular area to the edge servers of that area. This caching will make the content streaming faster by reducing the latency which will directly improve the user experience.
4. Application-Awareness and Content Optimization: A MEC server resides at the edge of the server which is closer to the radio network and the end-user, it can provide information about the application performance. A MEC platform can be used to provide application-aware performance optimization in near real-time by altering the encoding of streaming content inside the edge [121]. A video streaming can be optimized at the MEC server based on network resource availability, the screen size of MDs, etc. which helps in increasing the efficiency of network resource usages and the quality of experiences of end users [14]. Another example of application-awareness and content optimization is to use social networks as valuable sensors of human activity [122]. Information collected from social networks can be useful for optimizing the resource allocation for specific applications based on indicated popularity in a particular region.
5. Connected Vehicles: The connected vehicle technology can enlarge safety, reduce traffic congestion, sense vehicle behaviors, and provide opportunities for diverse value-added services such as the car finder and parking location [123,124]. However, this technology is still in a nascent stage and one of the main challenges is to satisfy the shortcoming is the latency requirement with the existing connected car clouds, which contributes to an end-to-end latency between 100 ms to 1s [16]. A MEC platform can be used as a key enabling technology for connected vehicles by adding computation and geo-distributed services to roadside BSs [16]. The services deployed in the MEC server

¹ <https://www.netflix.com/>.

² <https://www.primevideo.com/>.

can receive and analyze messages from proximate vehicles and roadside sensors and can propagate the hazard warnings and latency-sensitive messages rapidly which allows the drivers to react immediately and make it possible for autonomous driving. The tech giant Google had already announced its plan to develop self-driving vehicles back in 2009 [125].

6. Industrial Internet-of-Things (Industry 4.0): The fourth industrial revolution defines the next generation of manufacturing technology where information technologies are utilized in almost all operations to provide self optimization and automation for production, referred to as smart factory [126]. In a smart factory environment, all the components of the production system and auxiliary machinery, named industrial Internet-of-Things (IIoT) devices, are equipped with internet connection [127]. Based on the information provided by the IIoT devices, the smart manufacturing applications return management and control commands to the IIoT devices and these devices act on the command provided by the smart manufacturing applications. To achieve the optimal result, the smart factory rigorously requires ultra-low latency and a reliable working environment to acquire the precise control commands from the smart manufacturing applications for operations. Due to the low latency requirement of IIoT, the MEC platform can be a great fit to host smart-manufacturing applications.

6. Discussion

6.1. Technical challenges

Recent years have witnessed substantial research efforts on task offloading for MEC as surveyed in the preceding sections. However, there exists ample scopes for further research in various emerging directions centered around MEC task offloading problems that are still largely uncharted. In this section, we are discussing technical challenges towards solving the MEC task offloading problem.

- UEs and MEC server connection management: If a UE offloads a task to a MEC server attached to the nearest BS/SCs and before receiving the task computation result, that UE connects to a different MEC server of different BS/SCs. It is a key design issue to manage the connection between UEs and MEC servers to make sure UEs receive the task computation result in the scenario mentioned above.
- Estimation of computation time of a task: A decision on task offloading depends on several factors like computation time of requirements of a task, channel bandwidth, the proximity of MEC server, and many others. For task offloading decision to make the UEs have to calculate the approximate computation time requirements of the task. The offloading decision-making algorithm will be more efficient if a UE can calculate the most accurate computation time requirement of the task.
- Distributed MEC servers with multiple base stations or small cell base stations: Authors in [44] have proposed a distributed model, but the MEC servers are managed by a central SDN controller. So it is not a purely distributed model as the SDN controller is a centralized one. If the central SDN controller fails, the MEC server topology will be in a bad state as UEs will not receive any information from the SDN controller which is used for making task offloading decisions. This means there is no fault tolerance for the SDN controller. It is challenging to create a truly distributed system of MEC servers hosted across multiple BSs.
- Resource allocation for a task in MEC servers: It is important to allocate computation and storage resources to an application running on the MEC server to handle the task offloaded by UEs. Resources for an application can be allocated in various ways by initiating VMs, Docker containers, or any other technologies.

There is no standardization about the allocation of resources for different applications running inside the MEC servers to the best of our knowledge. It is challenging to devise a technique to allocate resources so that the resource allocation is efficient and at the same time it satisfies the resource requirements of a task.

- Task Scheduling: As task scheduling is an integral part of the task offloading framework, so an efficient task scheduling algorithm is needed. Once a task is offloaded to MEC servers, it is important to schedule the task for one of the application processes. As the nature task scheduling algorithm is NP-hard, it is a huge challenge to formulate an efficient task scheduling algorithm.
- The decision of task offloading to which MEC server or base station is a challenging task as i. the nearest base station may not host the MEC servers, ii. it may not have enough resources to serve the request, or iii. the requested application may not be available. As most of the tasks offloaded to MEC servers are delay-sensitive, we need to decide concisely which MEC server or base station to offload the tasks.

6.2. Future research direction

As research on task offloading to MEC is still in evolving stage, there are ample opportunities for research in this area.

1. Though there is a good progress in the usage of machine learning (ML) [50] for decision making in task offloading to MEC, it is a quite challenging task to formulate a machine learning model which takes into consideration of following attributes a. data rate between MDs and the BS which hosts MEC servers, b. application type of the task to be offloaded, c. computational capabilities and the energy status of the MDs, d. resource availability of the nearest BS which hosts the MEC servers, e. channel quality information between MDs and the BS, f. mobility of the MD devices etc. We can proceed in the direction of modeling a machine learning algorithm that considers the above attributes.
2. If we consider a MEC system model of single BS with multiple MEC servers attached to it and there is no cooperation among the BSs. A decision on task load balancing or distribution among MEC servers depends on many factors like types of the application running in different servers, resource utilization of these servers, application type of tasks and their different attributes, etc. In [44], authors have discussed task distribution among MEC servers based on a centralized decision-maker SDN controller, but as per our research, there is no distributed algorithm to distribute tasks among the servers inside a BS in an efficient way.
3. There is little research on task offloading for a distributed MEC server system model that includes multiple BSs. A BS can forward a task to another BS if it cannot satisfy the requirement of a task offloaded to it by a UE. A BS can forward a task to another BS for different reasons such as application needed by the UE is not available, resource constraints in the MEC servers, etc. We cannot just blindly forward a task to neighbor BS, we have to devise an efficient algorithm to forward the task to the nearest base station which has the required application and sufficient resources to serve the UEs. This is one of the open areas we can focus on.
4. An efficient task placement mechanism in the MEC is also important to satisfy the task offloading criteria like delay, computational resource need, and application type constraints. We need to focus on this area to formulate an efficient task placement mechanism that satisfies all the task offloading constraints.
5. After reviewing the literature on task offloading on MEC, we can firmly claim that impressive progress is happening in this field corresponding to distributed and centralized MEC systems. But we could not find much research progress in the area of the decentralized MEC systems model. Task offloading in decentralized MEC systems model could be one of the research areas we can focus on in the future.

7. Conclusion

The advancement of new types of delay-sensitive applications like face recognition augmented/virtual reality etc. needs a significant amount of computation power. Since both the requirements are not able to be addressed by either local computation or cloud computing, as a result a new network paradigm MEC is gaining popularity. One of the critical parts of the MEC is the efficient task offloading scheme. Though research in this area is still in the early stage, we have tried our best to provide a comprehensive survey of research progress in this field. First, we have divided the different task offloading models into three broad categories and conducted an overall literature review on recent research efforts on task offloading for MEC. We further discussed several challenges regarding task offloading and provided some potential future research direction in this area. The extensive overview and research outlook on task offloading in MEC provided in this survey hopefully can serve as handy references and valuable guidelines for further in-depth investigations of this area.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- [1] P. Porambage, J. Okwuibe, M. Liyanage, M. Ylianttila, T. Taleb, Survey on multi-access edge computing for internet of things realization, *IEEE Commun. Surv. Tutor.* 20 (4) (2018) 2961–2991.
- [2] J. Lee, J.-W. Kim, J. Lee, Mobile personal multi-access edge computing architecture composed of individual user devices, *Appl. Sci.* 10 (13) (2020) 4643.
- [3] Z. Sanaei, S. Abolfazli, A. Gani, R. Buyya, Heterogeneity in mobile cloud computing: taxonomy and open challenges, *IEEE Commun. Surv. Tutor.* 16 (1) (2013) 369–392.
- [4] B. Liu, X. Xu, L. Qi, Q. Ni, W. Dou, Task scheduling with precedence and placement constraints for resource utilization improvement in multi-user MEC environment, *J. Syst. Archit.* 114 (2021) 101970.
- [5] P. Ranaweera, A.D. Jurcut, M. Liyanage, Realizing multi-access edge computing feasibility: Security perspective, in: 2019 IEEE Conference on Standards for Communications and Networking, CSCN, IEEE, 2019, pp. 1–7.
- [6] Y.C. Hu, M. Patel, D. Sabella, N. Sprecher, V. Young, Mobile edge computing—A key technology towards 5G, ETSI White Pap. 11 (11) (2015) 1–16.
- [7] X. Xu, B. Shen, X. Yin, M.R. Khosravi, H. Wu, L. Qi, S. Wan, Edge server quantification and placement for offloading social media services in industrial cognitive IoT, *IEEE Trans. Ind. Inf.* (2020).
- [8] H. Dai, X. Zeng, Z. Yu, T. Wang, A scheduling algorithm for autonomous driving tasks on mobile edge computing servers, *J. Syst. Archit.* 94 (2019) 14–23.
- [9] X. Deng, J. Li, E. Liu, H. Zhang, Task allocation algorithm and optimization model on edge collaboration, *J. Syst. Archit.* 110 (2020) 101778.
- [10] C. Dou, S. Zhang, H. Wang, L. Sun, Y. Huang, W. Yue, Adhd fmri short-time analysis method for edge computing based on multi-instance learning, *J. Syst. Archit.* 111 (2020) 101834.
- [11] P. Cong, J. Zhou, L. Li, K. Cao, T. Wei, K. Li, A survey of hierarchical energy optimization for mobile edge computing: A perspective from end devices to the cloud, *ACM Comput. Surv.* 53 (2) (2020) 1–44.
- [12] C. Wu, Q. Peng, Y. Xia, Y. Ma, W. Zheng, H. Xie, S. Pang, F. Li, X. Fu, X. Li, et al., Online user allocation in mobile edge computing environments: A decentralized reactive approach, *J. Syst. Archit.* 113 (2021) 101904.
- [13] J.-W. Ryu, Q.-V. Pham, H.N. Luan, W.-J. Hwang, J.-D. Kim, J.-T. Lee, Multi-access edge computing empowered heterogeneous networks: A novel architecture and potential works, *Symmetry* 11 (7) (2019) 842.
- [14] T. Taleb, K. Samdanis, B. Mada, H. Flinck, S. Dutta, D. Sabella, On multi-access edge computing: A survey of the emerging 5G network edge cloud architecture and orchestration, *IEEE Commun. Surv. Tutor.* 19 (3) (2017) 1657–1681.
- [15] H. Guo, J. Liu, Collaborative computation offloading for multiaccess edge computing over fiber–wireless networks, *IEEE Trans. Veh. Technol.* 67 (5) (2018) 4514–4526.
- [16] Y. Mao, C. You, J. Zhang, K. Huang, K.B. Letaief, A survey on mobile edge computing: The communication perspective, *IEEE Commun. Surv. Tutor.* 19 (4) (2017) 2322–2358.
- [17] Q.-V. Pham, L.B. Le, S.-H. Chung, W.-J. Hwang, Mobile edge computing with wireless backhaul: Joint task offloading and resource allocation, *IEEE Access* 7 (2019) 16444–16459.
- [18] Y. Pochet, L.A. Wolsey, Production Planning By Mixed Integer Programming, Springer Science & Business Media, 2006.
- [19] P. Brucker, Scheduling Algorithms, fifth ed., Springer, 2010, ISBN: 3642089070, 9783642089077.
- [20] P. Salot, A survey of various scheduling algorithm in cloud computing environment, *Int. J. Res. Eng. Technol.* 2 (2) (2013) 131–135.
- [21] X. Lin, Y. Wang, Q. Xie, M. Pedram, Task scheduling with dynamic voltage and frequency scaling for energy minimization in the mobile cloud computing environment, *IEEE Trans. Serv. Comput.* 8 (2) (2014) 175–186.
- [22] T.Q. Dinh, J. Tang, Q.D. La, T.Q. Quek, Offloading in mobile edge computing: Task allocation and computational frequency scaling, *IEEE Trans. Commun.* 65 (8) (2017) 3571–3584.
- [23] H. Wu, W.J. Knottenbelt, K. Wolter, An efficient application partitioning algorithm in mobile environments, *IEEE Trans. Parallel Distrib. Syst.* 30 (7) (2019) 1464–1480.
- [24] Y.C. Hu, M. Patel, D. Sabella, N. Sprecher, V. Young, Mobile edge computing—A key technology towards 5G, ETSI White Pap. 11 (11) (2015) 1–16.
- [25] W. Shi, S. Dustdar, The promise of edge computing, *Computer* 49 (5) (2016) 78–81.
- [26] O. Salman, I. Elhajj, A. Kayssi, A. Chehab, Edge computing enabling the Internet of Things, in: 2015 IEEE 2nd World Forum on Internet of Things, WF-IoT, IEEE, 2015, pp. 603–608.
- [27] H. Tanaka, M. Yoshida, K. Mori, N. Takahashi, Multi-access edge computing: A survey, *J. Inf. Process.* 26 (2018) 87–97.
- [28] N. Abbas, Y. Zhang, A. Taherkordi, T. Skeie, Mobile edge computing: A survey, *IEEE Internet Things J.* 5 (1) (2017) 450–465.
- [29] Q.-V. Pham, F. Fang, V.N. Ha, M.J. Piran, M. Le, L.B. Le, W.-J. Hwang, Z. Ding, A survey of multi-access edge computing in 5G and beyond: Fundamentals, technology integration, and state-of-the-art, *IEEE Access* 8 (2020) 116974–117017.
- [30] Y. Mao, C. You, J. Zhang, K. Huang, K.B. Letaief, Mobile edge computing: Survey and research outlook, 2017, arXiv preprint arXiv:1701.01090.
- [31] S. Wang, X. Zhang, Y. Zhang, L. Wang, J. Yang, W. Wang, A survey on mobile edge networks: Convergence of computing, caching and communications, *IEEE Access* 5 (2017) 6757–6779.
- [32] P. Mach, Z. Becvar, Mobile edge computing: A survey on architecture and computation offloading, *IEEE Commun. Surv. Tutor.* 19 (3) (2017) 1628–1656.
- [33] B. Yang, X. Cao, J. Bassey, X. Li, L. Qian, Computation offloading in multi-access edge computing: A multi-task learning approach, *IEEE Trans. Mob. Comput.* (2020).
- [34] C. Jiang, X. Cheng, H. Gao, X. Zhou, J. Wan, Toward computation offloading in edge computing: A survey, *IEEE Access* 7 (2019) 131543–131558.
- [35] B. Wang, C. Wang, W. Huang, Y. Song, X. Qin, A survey and taxonomy on task offloading for edge-cloud computing, *IEEE Access* 8 (2020) 186080–186101.
- [36] H. Lin, S. Zeadally, Z. Chen, H. Labiod, L. Wang, A survey on computation offloading modeling for edge computing, *J. Netw. Comput. Appl.* (2020) 102781.
- [37] J. Wang, J. Pan, F. Esposito, P. Callyam, Z. Yang, P. Mohapatra, Edge cloud offloading algorithms: Issues, methods, and perspectives, *ACM Comput. Surv.* 52 (1) (2019) 1–23.
- [38] A. Shakarami, A. Shahidinejad, M. Ghobaei-Arani, A review on the computation offloading approaches in mobile edge computing: A game-theoretic perspective, *Softw. - Pract. Exp.* 50 (9) (2020) 1719–1759.
- [39] S. Thananjeyan, C.A. Chan, E. Wong, A. Nirmalathas, Mobility-aware energy optimization in hosts selection for computation offloading in multi-access edge computing, *IEEE Open J. Commun. Soc.* 1 (2020) 1056–1065.
- [40] X. Chen, Decentralized computation offloading game for mobile cloud computing, *IEEE Trans. Parallel Distrib. Syst.* 26 (4) (2014) 974–983.
- [41] T.X. Tran, D. Pompili, Joint task offloading and resource allocation for multi-server mobile-edge computing networks, *IEEE Trans. Veh. Technol.* 68 (1) (2018) 856–868.
- [42] X. Lyu, H. Tian, C. Sengul, P. Zhang, Multiuser joint task offloading and resource optimization in proximate clouds, *IEEE Trans. Veh. Technol.* 66 (4) (2016) 3435–3447.
- [43] I. ETSI, Multi-access edge computing (MEC); framework and reference architecture, Tech. rep., 2016, 2019.
- [44] M. Chen, Y. Hao, Task offloading for mobile edge computing in software defined ultra-dense network, *IEEE J. Sel. Areas Commun.* 36 (3) (2018) 587–597.
- [45] Z. Sun, M.R. Nakhai, An online learning algorithm for distributed task offloading in multi-access edge computing, *IEEE Trans. Signal Process.* (2020).
- [46] H.A. Alameddine, S. Sharafeddine, S. Sebbah, S. Ayoubi, C. Assi, Dynamic task offloading and scheduling for low-latency IoT services in multi-access edge computing, *IEEE J. Sel. Areas Commun.* 37 (3) (2019) 668–682.
- [47] H. Guo, J. Liu, J. Zhang, Computation offloading for multi-access mobile edge computing in ultra-dense networks, *IEEE Commun. Mag.* 56 (8) (2018) 14–19.
- [48] W. Ni, H. Tian, X. Lyu, S. Fan, Service-dependent task offloading for multiuser mobile edge computing system, *Electron. Lett.* 55 (15) (2019) 839–841.
- [49] T.T. Vu, N. Van Huynh, D.T. Hoang, D.N. Nguyen, E. Dutkiewicz, Offloading energy efficiency with delay constraint for cooperative mobile edge computing networks, in: 2018 IEEE Global Communications Conference, GLOBECOM, IEEE, 2018, pp. 1–6.

- [50] J. Wang, J. Hu, G. Min, A.Y. Zomaya, N. Georgalas, Fast adaptive task offloading in edge computing based on meta reinforcement learning, *IEEE Trans. Parallel Distrib. Syst.* 32 (1) (2020) 242–253.
- [51] F. Wei, S. Chen, W. Zou, A greedy algorithm for task offloading in mobile edge computing system, *China Commun.* 15 (11) (2018) 149–157.
- [52] T. Yang, R. Chai, L. Zhang, Latency optimization-based joint task offloading and scheduling for multi-user MEC system, in: 2020 29th Wireless and Optical Communications Conference, WOCC, IEEE, 2020, pp. 1–6.
- [53] M.D. Hossain, L.N. Huynh, T. Sultana, T.D. Nguyen, J.H. Park, C.S. Hong, E.-N. Huh, Collaborative task offloading for overloaded mobile edge computing in small-cell networks, in: 2020 International Conference on Information Networking, ICOIN, IEEE, 2020, pp. 717–722.
- [54] H. Yu, Q. Wang, S. Guo, Energy-efficient task offloading and resource scheduling for mobile edge computing, in: 2018 IEEE International Conference on Networking, Architecture and Storage, NAS, IEEE, 2018, pp. 1–4.
- [55] Z. Ning, P. Dong, X. Kong, F. Xia, A cooperative partial computation offloading scheme for mobile edge computing enabled Internet of Things, *IEEE Internet Things J.* 6 (3) (2018) 4804–4814.
- [56] T.Q. Dinh, J. Tang, Q.D. La, T.Q. Quek, Offloading in mobile edge computing: Task allocation and computational frequency scaling, *IEEE Trans. Commun.* 65 (8) (2017) 3571–3584.
- [57] C. You, K. Huang, H. Chae, B.-H. Kim, Energy-efficient resource allocation for mobile-edge computation offloading, *IEEE Trans. Wireless Commun.* 16 (3) (2016) 1397–1411.
- [58] J. Ren, G. Yu, Y. Cai, Y. He, Latency optimization for resource allocation in mobile-edge computation offloading, *IEEE Trans. Wireless Commun.* 17 (8) (2018) 5506–5519.
- [59] L. Yang, H. Zhang, M. Li, J. Guo, H. Ji, Mobile edge computing empowered energy efficient task offloading in 5G, *IEEE Trans. Veh. Technol.* 67 (7) (2018) 6398–6409.
- [60] S. Chouhan, Energy optimal partial computation offloading framework for mobile devices in multi-access edge computing, in: 2019 International Conference on Software, Telecommunications and Computer Networks, SoftCOM, IEEE, 2019, pp. 1–6.
- [61] T. Alfakih, M.M. Hassan, A. Gumaei, C. Savaglio, G. Fortino, Task offloading and resource allocation for mobile edge computing by deep reinforcement learning based on SARSA, *IEEE Access* 8 (2020) 54074–54084.
- [62] C. Yang, Y. Liu, X. Chen, W. Zhong, S. Xie, Efficient mobility-aware task offloading for vehicular edge computing networks, *IEEE Access* 7 (2019) 26652–26664.
- [63] G. Qiao, S. Leng, K. Zhang, Y. He, Collaborative task offloading in vehicular edge multi-access networks, *IEEE Commun. Mag.* 56 (8) (2018) 48–54.
- [64] J. Zhang, H. Guo, J. Liu, Y. Zhang, Task offloading in vehicular edge computing networks: A load-balancing solution, *IEEE Trans. Veh. Technol.* 69 (2) (2019) 2092–2104.
- [65] Y. Sun, X. Guo, J. Song, S. Zhou, Z. Jiang, X. Liu, Z. Niu, Adaptive learning-based task offloading for vehicular edge computing systems, *IEEE Trans. Veh. Technol.* 68 (4) (2019) 3061–3074.
- [66] P. Liu, J. Li, Z. Sun, Matching-based task offloading for vehicular edge computing, *IEEE Access* 7 (2019) 27628–27640.
- [67] H. Guo, J. Zhang, J. Liu, FiWi-enhanced vehicular edge computing networks: Collaborative task offloading, *IEEE Veh. Technol. Mag.* 14 (1) (2018) 45–53.
- [68] B. Gu, Z. Zhou, Task offloading in vehicular mobile edge computing: A matching-theoretic framework, *IEEE Veh. Technol. Mag.* 14 (3) (2019) 100–106.
- [69] H. Guo, J. Liu, J. Ren, Y. Zhang, Intelligent task offloading in vehicular edge computing networks, *IEEE Wirel. Commun.* 27 (4) (2020) 126–132.
- [70] Y. Wang, P. Lang, D. Tian, J. Zhou, X. Duan, Y. Cao, D. Zhao, A game-based computation offloading method in vehicular multiaccess edge computing networks, *IEEE Internet Things J.* 7 (6) (2020) 4987–4996.
- [71] S. Raza, W. Liu, M. Ahmed, M.R. Anwar, M.A. Mirza, Q. Sun, S. Wang, An efficient task offloading scheme in vehicular edge computing, *J. Cloud Comput.* 9 (2020) 1–14.
- [72] H. Peng, Q. Ye, X.S. Shen, SDN-based resource management for autonomous vehicular networks: A multi-access edge computing approach, *IEEE Wirel. Commun.* 26 (4) (2019) 156–162.
- [73] X. Wang, T. Wei, L. Kong, L. He, F. Wu, G. Chen, ECASS: Edge computing based auxiliary sensing system for self-driving vehicles, *J. Syst. Archit.* 97 (2019) 258–268.
- [74] J. Chen, S. Chen, S. Luo, Q. Wang, B. Cao, X. Li, An intelligent task offloading algorithm (iTOA) for UAV edge computing network, *Digit. Commun. Netw.* 6 (4) (2020) 433–443.
- [75] J. Xiong, H. Guo, J. Liu, Task offloading in UAV-aided edge computing: Bit allocation and trajectory optimization, *IEEE Commun. Lett.* 23 (3) (2019) 538–541.
- [76] Q. Wang, Z. Li, K. Nai, Y. Chen, M. Wen, Dynamic resource allocation for jointing vehicle-edge deep neural network inference, *J. Syst. Archit.* 117 (2021) 102133.
- [77] Z. Cao, P. Zhou, R. Li, S. Huang, D. Wu, Multiagent deep reinforcement learning for joint multichannel access and task offloading of mobile-edge computing in industry 4.0, *IEEE Internet Things J.* 7 (7) (2020) 6201–6213.
- [78] K. Zhang, Y. Zhu, S. Leng, Y. He, S. Maharjan, Y. Zhang, Deep learning empowered task offloading for mobile edge computing in urban informatics, *IEEE Internet Things J.* 6 (5) (2019) 7635–7647.
- [79] L. Huang, X. Feng, C. Zhang, L. Qian, Y. Wu, Deep reinforcement learning-based joint task offloading and bandwidth allocation for multi-user mobile edge computing, *Digit. Commun. Netw.* 5 (1) (2019) 10–17.
- [80] J. Wang, J. Hu, G. Min, W. Zhan, Q. Ni, N. Georgalas, Computation offloading in multi-access edge computing using a deep sequential model based on reinforcement learning, *IEEE Commun. Mag.* 57 (5) (2019) 64–69.
- [81] W. Tang, X. Zhao, W. Rafique, L. Qi, W. Dou, Q. Ni, An offloading method using decentralized P2P-enabled mobile edge servers in edge computing, *J. Syst. Archit.* 94 (2019) 1–13.
- [82] Z. Chen, X. Wang, Decentralized computation offloading for multi-user mobile edge computing: A deep reinforcement learning approach, *EURASIP J. Wireless Commun. Networking* 2020 (1) (2020) 1–21.
- [83] J.L. King, Centralized versus decentralized computing: Organizational considerations and management options, *ACM Comput. Surv.* 15 (4) (1983) 319–349.
- [84] A. Banerjee, M.S. Chishti, A. Rahman, R. Chanazain, Centralized framework for workload distribution in fog computing, in: 2018 3rd International Conference for Convergence in Technology, I2CT, IEEE, 2018, pp. 1–5.
- [85] A. Bozorgchenani, D. Tarchi, G.E. Corazza, Centralized and distributed architectures for energy and delay efficient fog network-based edge computing services, *IEEE Trans. Green Commun. Netw.* 3 (1) (2018) 250–263.
- [86] A.J. Ferrer, J.M. Marquès, J. Jorba, Towards the decentralised cloud: Survey on approaches and challenges for mobile, ad hoc, and edge computing, *ACM Comput. Surv.* 51 (6) (2019) 1–36.
- [87] B. Pon, Blockchain will usher in the era of decentralised computing, *LSE Bus. Rev.* (2016).
- [88] T.P. Lillicrap, J.J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, D. Wierstra, Continuous control with deep reinforcement learning, 2015, arXiv preprint arXiv:1509.02971.
- [89] H. Atiya, J. Welch, Distributed Computing: Fundamentals, Simulations, and Advanced Topics, Vol. 19, John Wiley & Sons, 2004.
- [90] J. Waldo, G. Wyant, A. Wollrath, S. Kendall, A note on distributed computing, in: International Workshop on Mobile Object Systems, Springer, 1996, pp. 49–64.
- [91] K. Nadiminti, M.D. De Assunção, R. Buyya, Distributed systems and recent innovations: Challenges and benefits, *InfoNet Mag.* 16 (3) (2006) 1–5.
- [92] Z. Kuang, Z. Ma, Z. Li, X. Deng, Cooperative computation offloading and resource allocation for delay minimization in mobile edge computing, *J. Syst. Archit.* (2021) 102167.
- [93] B. Gu, Z. Zhou, S. Mumtaz, V. Frasca, A.K. Bashir, Context-aware task offloading for multi-access edge computing: Matching with externalities, in: 2018 IEEE Global Communications Conference, GLOBECOM, IEEE, 2018, pp. 1–6.
- [94] T.-Y. Kan, Y. Chiang, H.-Y. Wei, Task offloading and resource allocation in mobile-edge computing system, in: 2018 27th Wireless and Optical Communication Conference, WOCC, IEEE, 2018, pp. 1–4.
- [95] J. Luo, X. Deng, H. Zhang, H. Qi, QoE-driven computation offloading for edge computing, *J. Syst. Archit.* 97 (2019) 34–39.
- [96] K. Benzekki, A. El Fergougui, A. Elbelhiti Elalaoui, Software-defined networking (SDN): a survey, *Secur. Commun. Netw.* 9 (18) (2016) 5803–5833.
- [97] E. Haleplidis, K. Pentikousis, S. Denazis, J.H. Salim, D. Meyer, O. Koufopavlou, Software-defined networking (SDN): Layers and architecture terminology, in: RFC 7426, IRTF, 2015.
- [98] Z.W. Geem, J.H. Kim, G.V. Loganathan, A new heuristic optimization algorithm: harmony search, *Simulation* 76 (2) (2001) 60–68.
- [99] A. Schrijver, Theory of Linear and Integer Programming, John Wiley & Sons, 1998.
- [100] C.H. Papadimitriou, K. Steiglitz, Combinatorial Optimization: Algorithms and Complexity, Courier Corporation, 1998.
- [101] Z. Kuang, L. Li, J. Gao, L. Zhao, A. Liu, Partial offloading scheduling and power allocation for mobile edge computing systems, *IEEE Internet Things J.* 6 (4) (2019) 6774–6785.
- [102] L.A. Wolsey, Integer Programming, Vol. 52, John Wiley & Sons, 1998.
- [103] D. Jakhari, I. Kaur, Artificial intelligence, machine learning and deep learning: definitions and differences, *Clin. Exp. Dermatol.* 45 (1) (2020) 131–132.
- [104] P. Ongsulee, Artificial intelligence, machine learning and deep learning, in: 2017 15th International Conference on ICT and Knowledge Engineering, ICT&KE, IEEE, 2017, pp. 1–6.
- [105] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, 2017, arXiv preprint arXiv:1707.06347.
- [106] E.L. Lawler, D.E. Wood, Branch-and-bound methods: A survey, *Oper. Res.* 14 (4) (1966) 699–719.
- [107] Y. Liu, M.J. Lee, An effective dynamic programming offloading algorithm in mobile cloud computing system, in: 2014 IEEE Wireless Communications and Networking Conference, WCNC, IEEE, 2014, pp. 1868–1873.

- [108] H.A. Lagar-Cavilla, N. Tolia, E. De Lara, M. Satyanarayanan, D. O'Hallaron, Interactive resource-intensive applications made easy, in: ACM/IFIP/USENIX International Conference on Distributed Systems Platforms and Open Distributed Processing, Springer, 2007, pp. 143–163.
- [109] D. Huang, P. Wang, D. Niyato, A dynamic offloading algorithm for mobile computing, *IEEE Trans. Wireless Commun.* 11 (6) (2012) 1991–1995.
- [110] N. Takahashi, H. Tanaka, R. Kawamura, Analysis of process assignment in multi-tier mobile cloud computing and application to edge accelerated web browsing, in: 2015 3rd IEEE International Conference on Mobile Cloud Computing, Services, and Engineering, IEEE, 2015, pp. 233–234.
- [111] Y. Nesterov, A. Nemirovskii, Interior-Point Polynomial Algorithms in Convex Programming, SIAM, 1994.
- [112] K.G. Murty, S.N. Kabadi, Some NP-Complete Problems in Quadratic and Nonlinear Programming, Tech. rep., 1985.
- [113] S. Sahni, Computationally related problems, *SIAM J. Comput.* 3 (4) (1974) 262–279.
- [114] J. Yang, Z. Lu, J. Wu, Smart-toy-edge-computing-oriented data exchange based on blockchain, *J. Syst. Archit.* 87 (2018) 36–48.
- [115] Y. Ai, M. Peng, K. Zhang, Edge computing technologies for Internet of Things: a primer, *Digit. Commun. Netw.* 4 (2) (2018) 77–86.
- [116] D. Sabella, A. Reznik, R. Frazao, Multi-Access Edge Computing in Action, CRC Press, 2019.
- [117] A. Anjum, T. Abdullah, M. Tariq, Y. Baltaci, N. Antonopoulos, Video stream analysis in clouds: An object detection and classification framework for high performance video analytics, *IEEE Trans. Cloud Comput.* (2016).
- [118] B. Varghese, N. Wang, S. Barbhuiya, P. Kilpatrick, D.S. Nikolopoulos, Challenges and opportunities in edge computing, in: 2016 IEEE International Conference on Smart Cloud, SmartCloud, IEEE, 2016, pp. 20–26.
- [119] J. Wang, J. Pan, F. Esposito, Elastic urban video surveillance system using edge computing, in: Proceedings of the Workshop on Smart Internet of Things, 2017, pp. 1–6.
- [120] S. Wang, M. Binstin, A smart transmission optimization mechanism for sports events based on edge computing, *Internet Technol. Lett.* 4 (1) (2021) e174.
- [121] M.A. Hoque, M. Siekkinen, J.K. Nurminen, Energy efficient multimedia streaming to mobile devices—A survey, *IEEE Commun. Surv. Tutor.* 16 (1) (2012) 579–597.
- [122] P. Garcia Lopez, A. Montresor, D. Epema, A. Datta, T. Higashino, A. Iamnitchi, M. Barcellos, P. Felber, E. Riviere, Edge-Centric Computing: Vision and Challenges, ACM New York, NY, USA, 2015.
- [123] P. Papadimitratos, A. De La Fortelle, K. Evennsen, R. Brignolo, S. Cosenza, Vehicular communication systems: Enabling technologies, applications, and future outlook on intelligent transportation, *IEEE Commun. Mag.* 47 (11) (2009) 84–95.
- [124] N. Lu, N. Cheng, N. Zhang, X. Shen, J.W. Mark, Connected vehicles: Solutions and challenges, *IEEE Internet Things J.* 1 (4) (2014) 289–299.
- [125] L. Lin, X. Liao, H. Jin, P. Li, Computation offloading toward edge computing, *Proc. IEEE* 107 (8) (2019) 1584–1607.
- [126] N.-N. Dao, Y. Lee, S. Cho, E. Kim, K.-S. Chung, C. Keum, Multi-tier multi-access edge computing: The role for the fourth industrial revolution, in: 2017 International Conference on Information and Communication Technology Convergence, ICTC, IEEE, 2017, pp. 1280–1282.

- [127] M. Wollschlaeger, T. Sauter, J. Jasperneite, The future of industrial communication: Automation networks in the era of the internet of things and industry 4.0, *IEEE Ind. Electron. Mag.* 11 (1) (2017) 17–27.



Akhirul Islam received the Bachelor's (B.E.) degree in Computer Science and Engineering from Dibrugarh University, India in 2009 and currently pursuing Ph.D. from Indian Institute of Information Technology, Guwahati, India. He also has 11+ years of experience in software industry and working as a Staff Software Dev Engineer in FireEye, Bangalore, India R&D center. His present research focus is on Multi-access Edge Computing(MEC), specially in MEC task offloading.



Arindam Debnath received the Bachelor's (B.Tech.) degree in Information Technology from Gauhati University, India in 2009 and currently pursuing Master's degree (M.Tech.) from Indian Institute of Information Technology, Guwahati, India.



Manojit Ghose is working as an Assistant Professor in the Department of Computer Science and Engineering at IIIT Guwahati. Prior to joining IIIT Guwahati, he worked as an Assistant Professor at Dibrugarh University. He has completed his B.E. from Dibrugarh University, M.Tech. and Ph.D. from IIT Guwahati in Computer Science and Engineering. His research area broadly includes multiprocessor scheduling, cloud computing, and computer architecture. He is a member of IEEE and serves as TPC member for several conferences of repute.



Suchetana Chakraborty is currently working as an Assistant Professor at the department of Computer Science and Engineering, Indian Institute of Technology Jodhpur, India. Prior she was a faculty member at the department of CSE, IIIT Guwahati, Assam, India. Her research interests are broadly in the areas of distributed systems, ubiquitous computing, wireless sensor and ad hoc networks, edge computing and Internet of Vehicles. She received her M.Tech. and Ph.D. degree from Department of CSE, IIT Guwahati in the year 2011 and 2014, respectively. She is a member of IEEE, IEEE ComSoc, ACM and IMOBILE, the Indian chapter of ACM SIGMOBILE. She can be contacted at suchetana@iitj.ac.in.